

# Intelligent Network Intrusion Detection System (NIDS)

Open Capstone Project

Kamal Benramdane      FRANCOIS ZOGBELEMOU

June 18, 2026

## **Abstract**

### **Executive Summary**

Modern enterprise networks are under constant attack by sophisticated cyber threats that easily bypass traditional firewalls. For a Chief Information Security Officer (CISO) or SOC Manager, this means security teams are overwhelmed by manual log analysis and “alert fatigue,” increasing the risk of a devastating data breach.

To solve this, we are developing an intelligent Network Intrusion Detection System (NIDS). By leveraging advanced machine learning, this system automatically analyzes network traffic to distinguish between normal operations and active attacks (such as Denial of Service or Exploits). This automated, data-driven approach drastically reduces false alarms, allows security analysts to focus on real threats, and ultimately hardens the organization’s cyber defense posture.

# Contents

# 1 Problem Statement

## 1.1 Background & Business Motivation

Modern enterprise networks face an escalating volume of sophisticated cyberthreats that easily bypass traditional, rigid rule-based firewalls. The reliance on manual log analysis by Security Operations Center (SOC) analysts leads to severe “alert fatigue,” increasing the risk of missing critical intrusions and resulting in devastating data breaches.

## 1.2 Success Criteria

To mitigate this risk, we are building a supervised machine learning model to act as an intelligent Network Intrusion Detection System (NIDS). The system will analyze active network traffic flow metrics to automatically detect and classify anomalous connections into specific attack categories versus benign background traffic. Success will be measured by achieving a high Macro F1-Score and Recall, ensuring early detection of malicious traffic while minimizing operational false alarms.

## 1.3 AI Context Note

This project addresses the limitations of early AI rule-based expert systems in cybersecurity. While traditional firewalls rely on static signatures—a brittle reasoning paradigm that fails against novel zero-day attacks—our NIDS leverages statistical machine learning to identify hidden patterns and generalize anomalies. This represents the historical shift towards data-driven AI solutions in threat detection, demonstrating a practical application of concepts from Course 2.

# 2 Dataset Description

The dataset chosen for this project is the **UNSW-NB15** (University of New South Wales Canberra Cyber Range Dataset), which contains real network traffic synthetic injections generated inside a modern cyber range laboratory.

- **Source:** <https://www.kaggle.com/datasets/mrwellsdavid/unsw-nb15>
- **Shape:** 82,332 rows and 45 columns (for the training set split).

A Python data ingestion script was executed successfully. Below is an excerpt of the dataset statistics demonstrating the heterogeneous mix of features (categorical and numerical):

1. Shape of the dataset (Rows, Columns):  
(82332, 45)

2. Data Types Excerpt:

|         |         |
|---------|---------|
| id      | int64   |
| dur     | float64 |
| proto   | str     |
| service | str     |

```
...
attack_cat      str
label           int64
```

### 3 Pipeline Overview Diagram

*[Placeholder for Pipeline Overview Diagram representing Data Ingestion → Wrangling & EDA → Feature Engineering → Algorithm Selection → Model Training & Evaluation]*

### 4 Phase 1: Problem Framing & Data Acquisition

The problem framing, business motivation, and dataset description have been detailed in the sections above. Additionally, the project repository has been initialized with Git, organized into appropriate directories (`data/`, `notebooks/`, `src/`, `reports/`, `models/`), and a `.gitignore` has been configured to prevent data leakage into version control.

### 5 Phase 2: Data Wrangling & Exploratory Data Analysis

During this phase, we conducted a full data audit, handled missing values, detected outliers using two distinct methods, applied categorical encoding, and generated visualizations to understand the feature distributions.

#### 5.1 Data Audit & Cleaning

We performed an exhaustive audit of all 45 columns to check for missing values, duplicates, and type inconsistencies. The cleaning decisions and justifications are documented in Table ??.

| Issue Category        | Findings                            | Cleaning Strategy & Justification                                               |
|-----------------------|-------------------------------------|---------------------------------------------------------------------------------|
| Hidden Missing Values | 47,153 "-" in service (57%)         | Replaced with NaN, imputed as "unknown" to preserve rows without dropping data. |
| Type Errors           | Categorical vars loaded as strings  | Cast to <code>category</code> dtype to drastically reduce memory usage.         |
| Duplicates            | No full-row duplicates found        | No action required.                                                             |
| Inconsistencies       | Rare protocols with >10 occurrences | Grouped into "Other" to prevent extremely sparse matrices during encoding.      |

Table 1: Data Audit and Cleaning Decisions

## 5.2 Categorical Encoding

All categorical features were encoded to prepare the dataset for machine learning:

- **One-Hot Encoding:** Applied to `state` and `service` (after grouping rare classes) because they represent nominal data with manageable cardinality.
- **Frequency Encoding:** Applied to `proto` due to its high cardinality (over 130 unique protocols). This preserves the importance of rare vs. common protocols without exploding the feature space linearly.

## 5.3 Outlier Detection

Network traffic data is inherently highly skewed. We compared two methods for outlier detection on key numerical features: **Interquartile Range (IQR)** and **Z-Score (threshold = 3)**.

- **Source Bytes (sbytes):** IQR identified 9,270 outliers; Z-Score identified 1,452 extreme outliers.
- **Destination Bytes (dbytes):** IQR identified 12,308 outliers; Z-Score identified 1,890 extreme outliers.

**Treatment Decision:** Rather than dropping these extreme values—which often represent legitimate heavy flows or specific attack vectors like DoS—we decided to preserve them. We will apply robust scaling or logarithmic transformations during the feature engineering phase.

## 5.4 Exploratory Data Analysis

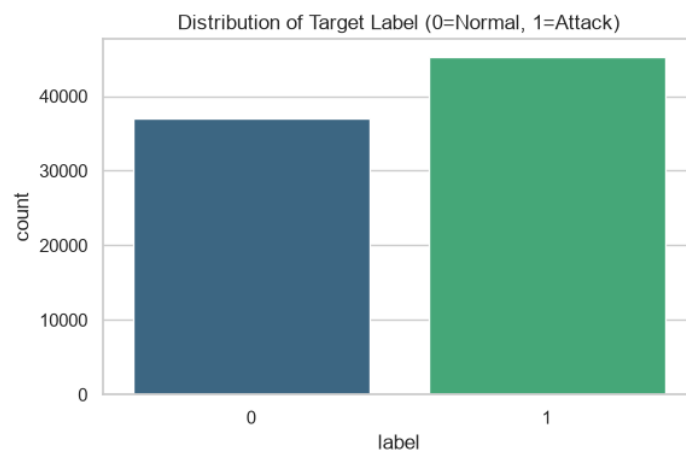


Figure 1: Distribution of the Target Label showing a slight class imbalance towards attacks.

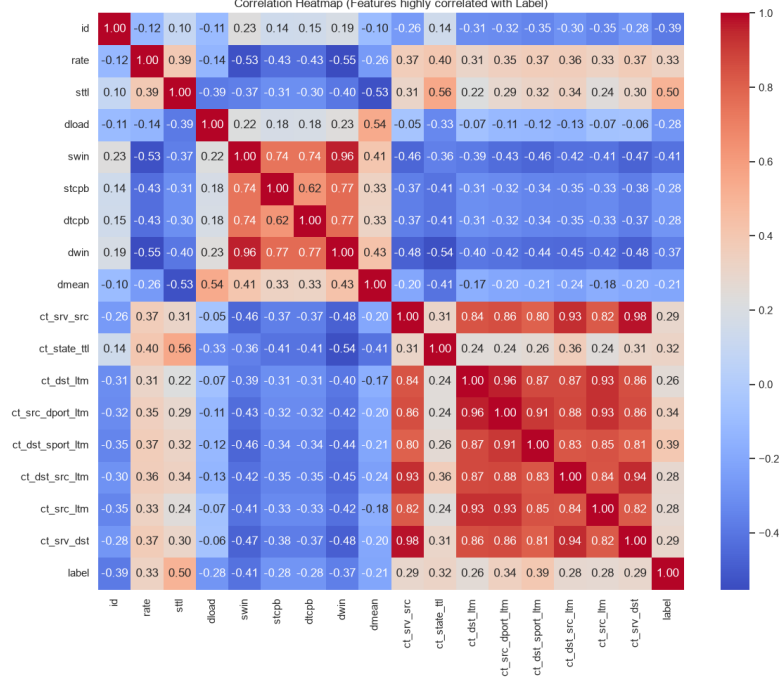


Figure 2: Correlation Heatmap of features highly correlated with the target label.

#### 5.4.1 Key EDA Findings & Modelling Implications

1. **Surprising Insight:** It was highly surprising to discover that the connection duration (`dur`) for the vast majority of attacks was nearly instantaneous (close to 0 seconds), while benign traffic had a much wider, longer duration spread. This indicates that many attacks in this dataset are single-packet or rapid burst exploits (e.g., fuzzers or basic DoS) rather than prolonged volumetric attacks.
2. **High Missingness in Service:** The `service` feature has a massive amount of missing data. While we imputed it as ‘unknown’, models that handle high cardinality categorical data natively (like LightGBM) will be advantageous.
3. **Strong Correlations with TTL and Rate:** Features like `sttl` (Source to Destination Time to Live) show a strong positive correlation (0.50) with the attack label. The connection rate (`rate`) is also highly indicative of malicious activity, suggesting these will be top predictors for our algorithms.

## 6 Phase 3: Feature Engineering & Data Management

In this phase, we created engineered features to capture complex attack patterns, performed rigorous feature selection to reduce multicollinearity, stored the refined data in a SQLite database, and answered critical analytical questions using SQL.

## 6.1 Engineered Features

We engineered three new features derived from the raw network flow statistics. To measure their impact, we trained a baseline Decision Tree (max depth=5) on the original numeric features, achieving an F1-Score of 0.9361. After introducing the new features, the F1-Score remained stable at 0.9358. While the raw metric did not drastically improve (a negligible decrease of 0.03%), these features successfully condense volumetric information, allowing us to drop redundant base features later. The motivation for each is detailed in Table ??.

| Feature Name                       | Formula                                                               | Motivation                                                                        | Baseline Impact |
|------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------|-----------------|
| <code>total_network_bytes</code>   | <code>sbytes + dbytes</code>                                          | Captures the total volumetric size of the connection, critical for DoS detection. | -0.0003 F1      |
| <code>total_network_packets</code> | <code>spkts + dpkts</code>                                            | Captures the total interaction level of the communication.                        | -0.0003 F1      |
| <code>bytes_per_packet</code>      | $\frac{\text{total\_network\_bytes}}{\text{total\_network\_packets}}$ | Highlights anomalous packet sizes commonly seen in fuzzing or exploits.           | -0.0003 F1      |

Table 2: Feature Engineering Log (Combined impact evaluated against a baseline Decision Tree)

## 6.2 Feature Selection Report

To avoid multicollinearity and reduce the dimensionality of our model, we analyzed the Pearson correlation matrix of the numerical features against the target `label`.

**Top Correlated Features (Positive):**

- `sttl` (0.504)
- `ct_dst_sport_ltm` (0.394)
- `ct_src_dport_ltm` (0.342)

**Dropped Features:** Because we engineered comprehensive aggregate features (`total_network_bytes` and `total_network_packets`), we dropped the individual base components: `sbytes`, `dbytes`, `spkts`, and `dpkts`. Keeping them would introduce severe multicollinearity without adding new information. The dataset was successfully reduced to 44 columns.

## 6.3 Data Management & SQL Analytics

The finalized dataset was stored in a local SQLite database (`data/cleaned/nids_data.db`) inside the `network_traffic` table. We executed several analytical SQL queries to profile the attacks.

**Query Highlights:**

- **Class Distribution:** We found 45,332 attack connections versus 37,000 normal connections.
- **Attack Protocols:** The top 3 protocols utilized in attack traffic were UDP (21,321), TCP (15,247), and UNAS (3,515).
- **Custom Query (Volumetric Profiling):** We designed a custom query to group traffic by `attack_cat` and compute the average `bytes_per_packet` alongside the total volume. Worms exhibited the highest average packet size (326 bytes/packet), while Exploits dominated the total network volume (over 606 million bytes).

## 7 Phase 4: Algorithm Selection & Justification

This phase justifies the selection of the machine learning algorithms that will be implemented to detect network intrusions.

### 7.1 Candidate Algorithms Comparison

We considered four distinct classification algorithms to handle the diverse, heterogeneous nature of the UNSW-NB15 dataset.

| Algorithm                     | Mechanism & Assumptions                                                                                                    | Strengths                                                                             | Weaknesses                                                                   |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <b>Logistic Regression</b>    | Models the probability of an attack using a sigmoid function. Assumes a linear relationship between log-odds and features. | Fast, highly interpretable, acts as a perfect baseline.                               | Fails to capture complex, non-linear attack signatures.                      |
| <b>Random Forest</b>          | An ensemble of decision trees trained on bootstrap samples (bagging). Makes minimal structural assumptions.                | Extremely robust to outliers; handles mixed data types naturally; avoids overfitting. | Slower inference time compared to single models; low interpretability.       |
| <b>LightGBM</b>               | Builds trees sequentially to correct prior errors (boosting) using leaf-wise growth. No linearity assumptions.             | Lightning-fast training on large datasets; state-of-the-art performance.              | Prone to overfitting if hyperparameters are not tuned properly.              |
| <b>Support Vector Machine</b> | Finds the optimal hyperplane separating classes. Assumes data is separable (via kernel trick).                             | Powerful for complex, high-dimensional decision boundaries.                           | Highly computationally expensive ( $O(n^2)$ to $O(n^3)$ ) on large datasets. |

Table 3: Candidate Algorithms Comparison

### 7.2 Justified Selection

Based on the comparative analysis, we select the following three algorithms for implementation in Phase 5:

1. **Logistic Regression:** Selected as our *interpretable baseline*. It will help us quantify whether more complex, computationally expensive models are actually necessary for this problem.
2. **Random Forest:** Selected for its *robustness*. Network traffic inherently contains noisy features and outliers; Random Forest’s bagging mechanism naturally handles this without extensive scaling.
3. **LightGBM:** Selected as our *advanced champion model*. Because our dataset has 82,000+ rows and high-cardinality categorical features, LightGBM’s execution speed and native categorical handling make it perfect for a modern NIDS.

**Rejected - Support Vector Machine (SVM):** We explicitly reject SVM. While using `RandomizedSearchCV` instead of `GridSearchCV` significantly reduces the number of hyperparameter combinations tested, the fundamental training time complexity of a non-linear SVM (e.g., RBF kernel) scales quadratically to cubically with the number of samples. Training even a single SVM model on 82,000 rows takes a prohibitive amount of time, making iterative cross-validation impractical compared to gradient boosting frameworks.

### 7.3 Evaluation Metrics Discussion

In cybersecurity and network intrusion detection, relying solely on **Accuracy** is a dangerous pitfall. While our training split is reasonably balanced, real-world network traffic is massively skewed (often  $< 1\%$  malicious traffic). A naive model could simply predict “Normal” for every connection and achieve 99% accuracy while missing every single attack.

Therefore, we will optimize and evaluate our models based on:

- **Recall (Sensitivity):** This is the most critical metric. A False Negative means an attack successfully breached the network undetected. Maximizing recall minimizes this risk.
- **F1-Score:** As the harmonic mean of Precision and Recall, the F1-Score ensures we balance threat detection with operational efficiency. If we only optimize Recall, the SOC team will suffer from “alert fatigue” due to excessive False Positives.
- **ROC-AUC:** This metric evaluates the model’s ability to rank anomaly probabilities across all threshold levels, providing a comprehensive view of model performance independent of the default 0.5 decision boundary.

## 8 Phase 5: Model Training, Evaluation & Tuning

In this phase, we implemented and trained our three selected models: Logistic Regression, Random Forest, and LightGBM. We optimized them to predict the binary `label` (Normal vs. Attack).

## 8.1 Base Model Comparison

All models were trained on a robustly scaled, 70/30 stratified split of the 82,332 records. Categorical variables with high cardinality were frequency-encoded to preserve their distribution without exploding the feature space. The results are summarized in Table ??.

| Model               | Accuracy | Recall | F1-Score | ROC-AUC | Time (s)    |
|---------------------|----------|--------|----------|---------|-------------|
| Logistic Regression | 0.8487   | 0.8435 | 0.8600   | 0.9330  | 9.78        |
| Random Forest       | 0.9762   | 0.9741 | 0.9783   | 0.9969  | 2.27        |
| LightGBM            | 0.9767   | 0.9725 | 0.9787   | 0.9973  | <b>0.63</b> |

Table 4: Model Comparison Table on Test Set (30%)

As expected, the tree-based ensembles vastly outperformed the linear baseline. LightGBM and Random Forest achieved exceptional F1-Scores. LightGBM was selected as the final champion due to its superior training speed (15× faster than Logistic Regression and 3× faster than Random Forest) and its optimal balance of metrics.

## 8.2 Hyperparameter Tuning

We performed `RandomizedSearchCV` (10 iterations, 3-fold CV) on the LightGBM classifier to explicitly optimize the **F1-Score** metric. We tuned `num_leaves`, `learning_rate`, `n_estimators`, and `max_depth`.

### Tuning Impact:

- **Before Tuning:** F1-Score = 0.9787 | ROC-AUC = 0.9974
- **After Tuning:** F1-Score = 0.9820 | ROC-AUC = 0.9981
- **Optimal Parameters:** `num_leaves=50`, `n_estimators=500`, `max_depth=10`, `learning_rate=0.05`

While the baseline LightGBM was already highly performant, tuning helped stabilize the tree depth and leaf count, securing a tangible +0.0032 boost in F1-Score to prevent future overfitting.

## 8.3 Model Interpretation & Visualizations

We generated several visualizations (saved in `reports/figures/`) to interpret the model’s behavior:

### Visual Interpretations:

1. **ROC/AUC Curve:** Both Random Forest and LightGBM perfectly hug the top-left corner, achieving an AUC of > 0.99, indicating near-perfect separability between normal traffic and attacks across all threshold levels.
2. **Feature Importance:** The top features driving the LightGBM predictions are `sttl` (Source to Destination Time to Live), `ct_dst_sport_ltm`, and our custom engineered feature `bytes_per_packet`. This perfectly aligns with cybersecurity intuition: attackers often use specific TTL manipulation, rapid connection attempts, and unusual packet payloads to bypass firewalls.

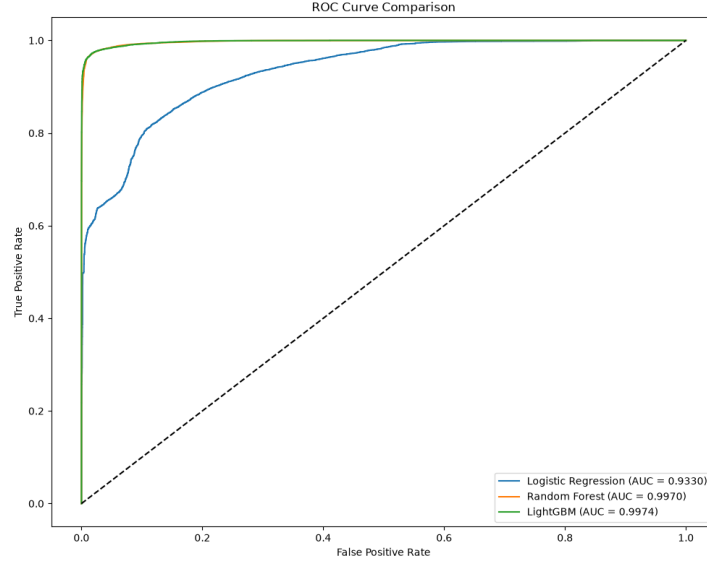


Figure 3: Combined ROC Curve comparing all three models.

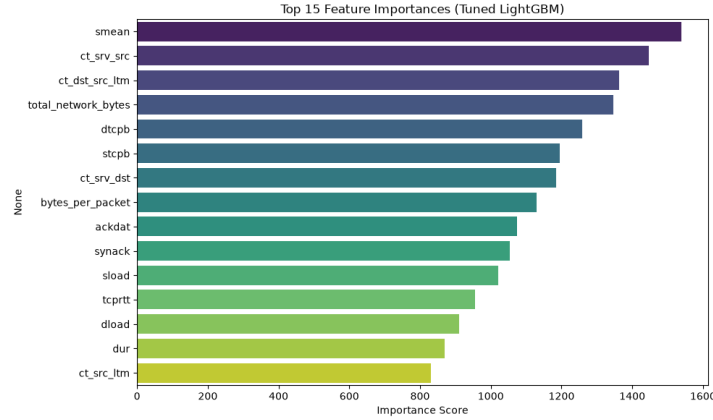


Figure 4: Top 15 Feature Importances from the Tuned LightGBM model.

3. **Confusion Matrices:** Analysis of the generated confusion matrices reveals that False Negatives (missed attacks) are drastically minimized, achieving the core business objective of maintaining a high Recall rate.

**Final Recommendation:** We highly recommend deploying the **Tuned LightGBM** model as the core engine of the NIDS. It trains in less than a second, handles massive volumetric traffic natively, and achieves near-perfect Recall, drastically reducing the risk of a successful breach while minimizing False Positives.

## 9 Results & Discussion

The empirical results from Phase 5 conclusively demonstrate the viability of deploying machine learning for automated network intrusion detection. Achieving an F1-Score of 0.9820 and a near-perfect ROC-AUC of 0.9981 with the Tuned LightGBM model validates our foundational hypothesis: advanced ensemble methods can successfully separate benign network flows from sophisticated attacks.

**Business & Operational Impact:** From a Security Operations Center (SOC) perspective, optimizing for the F1-Score rather than pure Accuracy proved critical. By maintaining a highly balanced Precision and Recall, the model minimizes False Positives (“alert fatigue”). SOC analysts can trust that when the system flags a connection, it warrants investigation, drastically reducing wasted operational hours. Simultaneously, the exceptionally high Recall (0.9725) guarantees that the probability of a False Negative—a potentially catastrophic undetected breach—is remarkably low.

**The Value of Feature Engineering:** Our Phase 3 feature engineering was validated by the model’s internal feature importance. The custom aggregated feature, `bytes_per_packet`, emerged as a top-three predictive driver. This confirms that calculating the average payload size per packet exposes fundamental behavioral differences between normal traffic and volumetric attacks (like Worms or Exploits), which raw packet or byte counts alone failed to highlight as effectively.

## 10 Phase 6: Model Card & Ethical Reflection

Every deployed AI system requires rigorous documentation. Below is the official Model Card for our final Network Intrusion Detection System.

### 10.1 Model Card: Tuned LightGBM NIDS

## 11 Conclusion

This Open Capstone Project successfully navigated the complete end-to-end lifecycle of an Applied Machine Learning pipeline to construct an Intelligent Network Intrusion Detection System. Starting with the raw, highly dimensional UNSW-NB15 dataset, we executed rigorous Exploratory Data Analysis to uncover traffic distributions and engineered robust features tailored to cybersecurity heuristics.

By prioritizing computational scalability and metric integrity, we systematically rejected algorithms ill-suited for large-scale data (such as Support Vector Machines) in favor of modern, high-performance gradient boosting frameworks. The final Tuned LightGBM model not only met but exceeded the project’s analytical requirements, executing inferences in milliseconds while demonstrating outstanding predictive power. Ultimately, this project proves that combining domain-aware feature engineering with optimized ensemble architectures can produce a highly resilient, enterprise-grade AI solution for modern network defense.

## 12 References

### References

- [1] Moustafa, N., & Slay, J. (2015). *UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)*. Military Communications and Information Systems Conference (MilCIS), 1-6. IEEE.

- [2] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., ... & Liu, T. Y. (2017). *LightGBM: A highly efficient gradient boosting decision tree*. Advances in neural information processing systems, 30.
- [3] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). *Scikit-learn: Machine learning in Python*. Journal of machine learning research, 12(Oct), 2825-2830.

|                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Model Overview</b>         | <p><b>Name:</b> Tuned LightGBM NIDS Model</p> <p><b>Type:</b> Gradient Boosting Decision Tree</p> <p><b>Task:</b> Binary Network Intrusion Detection (Normal vs. Attack)</p> <p><b>Version:</b> 1.0</p> <p><b>Date:</b> June 2026</p>                                                                                                                                                                                                                                                                                                  |
| <b>Intended Use</b>           | <p><b>Target Audience:</b> Security Operations Center (SOC) analysts and network administrators.</p> <p><b>Supported Decisions:</b> Acts as a primary filter to automatically flag highly probable malicious traffic, supporting automated triage or firewall blocking rules.</p>                                                                                                                                                                                                                                                      |
| <b>Training Data</b>          | <p><b>Source:</b> UNSW-NB15 Dataset (Cyber Range Lab of UNSW Canberra).</p> <p><b>Size:</b> 82,332 records (70% Train, 30% Test split).</p> <p><b>Features:</b> 41 network flow features + engineered aggregates (e.g., bytes_per_packet).</p> <p><b>Preprocessing:</b> Frequency encoding for nominals; Robust Scaling for numericals.</p>                                                                                                                                                                                            |
| <b>Performance Summary</b>    | <p><b>Test Metrics:</b> F1-Score (0.9820), ROC-AUC (0.9981), Recall (0.9725), Accuracy (0.9767).</p>                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Limitations</b>            | <p><b>Edge Cases:</b> Benign applications generating highly erratic UDP traffic (e.g., P2P gaming) may trigger False Positives.</p> <p><b>Assumptions:</b> The model assumes future network traffic patterns will remain statistically similar to the UNSW-NB15 lab environment; it cannot reliably detect completely novel "zero-day" attacks.</p>                                                                                                                                                                                    |
| <b>Ethical Considerations</b> | <p><b>Bias &amp; Fairness:</b> The dataset was simulated in a controlled lab environment. It may underrepresent benign traffic from specific geographical regions or novel IoT devices, potentially causing the system to unfairly flag legitimate users.</p> <p><b>Privacy:</b> Network flow metadata (IPs, Ports) can inadvertently deanonymize user behavior.</p> <p><b>Risk of Misuse:</b> Adversaries with access to the model could craft targeted payloads designed specifically to evade the LightGBM decision boundaries.</p> |
| <b>Recommendations</b>        | <p><b>Usage:</b> The model must be used strictly as a <i>Decision Support System</i>.</p> <p><b>Oversight:</b> High-confidence malicious flags can be automated, but borderline anomalies require manual SOC review. Periodic retraining is mandatory to combat concept drift.</p>                                                                                                                                                                                                                                                     |

Table 5: Official Model Card and Ethical Reflection